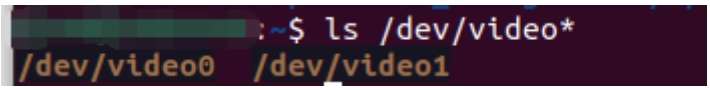
Barcode Recognition

- Barcode Recognition
  - Hardware Connection
  - Install Driver Libraries
  - Example Code
  - Running the Program
  - Program Output
  - Optional: Generating Barcodes

Hardware Connection

Plug the USB camera into the motherboard's USB port. Open the system terminal and enter the following command to check the camera's device number. If /dev/video0 appears, it's working correctly.

```
ls /dev/video*
```



Install Driver Libraries

```
sudo apt install libzbar-dev
sudo pip3 install pyzbar --break-system-packages
sudo pip3 install python-barcode --break-system-packages
```

Example Code

```
import cv2 as cv
from pyzbar.pyzbar import decode
import time
import numpy as np
from PIL import Image, ImageDraw, ImageFont


def decodeDisplay(image):
    gray = cv.cvtColor(image, cv.COLOR_BGR2GRAY)
    barcodes = decode(gray)
    for barcode in barcodes:
        # 提取条形码的边界框的位置
        # Extract the position of the boundary box of the barcode
        (x, y, w, h) = barcode.rect
        cv.rectangle(image, (x, y), (x + w, y + h), (225, 0, 0), 5)
        encoding = 'UTF-8'
        # 画出来，就需要先将它转换成字符串
        # to draw it, you need to convert it to a string
        barcodeData = barcode.data.decode(encoding)
        barcodeType = barcode.type
        # 绘出图像上数据和类型
```

```
        # Draw the data and type on the image
        pilimg = Image.fromarray(image)
        # 创建画笔
        # create brush
        draw = ImageDraw.Draw(pilimg)  # 图片上打印  Print on picture
        # # 参数1：打印坐标，参数2：文本，参数3：字体颜色
        # Parameter 1: print coordinates, parameter 2: text, parameter 3: font
color
        draw.text((x, y - 25), str(barcode.data, encoding), fill=(255, 0, 0))
        # # PIL图片转cv2 图片
        # PIL picture to CV2 picture
        image = cv.cvtColor(np.array(pilimg), cv.COLOR_RGB2BGR)
        # 向终端打印条形码数据和条形码类型
        # Print barcode data and barcode type to terminal
        print("[INFO] Found {} barcode: {}".format(barcodeType, barcodeData))
    return image


if __name__ == '__main__':
    capture = cv.VideoCapture(0)
    cv_edition = cv.__version__
    if cv_edition[0] == '3': capture.set(cv.CAP_PROP_FOURCC,
cv.VideoWriter_fourcc(*'XVID'))
    else: capture.set(cv.CAP_PROP_FOURCC, cv.VideoWriter.fourcc('M', 'J', 'P',
'G'))
    capture.set(cv.CAP_PROP_FRAME_WIDTH, 640)
    capture.set(cv.CAP_PROP_FRAME_HEIGHT, 480)
    print ("capture get FPS : ",capture.get(cv.CAP_PROP_FPS))
    while capture.isOpened():
        start = time.time()
        ret, frame = capture.read()
        action = cv.waitKey(10) & 0xFF
        frame = decodeDisplay(frame)
        end = time.time()
        fps = 1 / (end - start)
        text = "FPS : " + str(int(fps))
        cv.putText(frame, text, (30, 30), cv.FONT_HERSHEY_SIMPLEX, 0.6, (100,
200, 200), 1)
        cv.imshow('frame', frame)
        if action == ord('q') or action == 113: break
    capture.release()
    cv.destroyAllWindows()
```

Running the Program

Open the desktop terminal and execute the program:

```
cd ~/content_recognition/barcode
python3 barcode_parsing_usb.py
```

Note: Since the program requires a display window to show the video feed, do not run it via a remote SSH terminal. Please run the program using the native desktop environment or a VNC remote desktop session.

Program Output

Upon execution, the program automatically activates the USB camera. Simply point the camera at a barcode; the program will then automatically frame the barcode and print its information to the terminal.

Barcode Example:

Optional: Generating Barcodes

Open the terminal and run the following command to regenerate barcodes within the `code` directory.

```
cd ~/content_recognition/barcode
python3 barcode_create.py
```

content_recognition / barcode / code
ean13_1.
png
ean13_2.
png
ean13_3.
png